CDA and Enterprise BI

What communications data acquisition has in common with business intelligence and data analysis — and how to present prior experience as directly relevant

Prepared for Michael Rossi
October 2026

Contents

Contents 2

1. The same stack, different labels 4

2. Disciplines that transfer directly 5

2.1 Data cataloguing and classification 5

2.2 Master data, identity resolution and slowly changing dimensions 5

2.3 Data quality frameworks 5

2.4 Lineage and provenance 6

2.5 Access control: row-level and column-level security 6

2.6 Retention, archival and deletion 6

2.7 Operational reporting versus evidential reporting 6

2.8 Capacity planning and the thing everyone forgets 6

2.9 Vendor platforms versus owned policy 7

2.10 Requirements traceability and governance 7

3. What does not transfer — say this first 8

4. Presenting your own experience — solutions 9

4.1 Field-level classification with legal effect 9

4.2 Time-qualified identity resolution 9

4.3 Volume reconciliation and silent feed loss 10

4.4 Lineage and provenance for a scrutinised output 11

4.5 Access control per request, not per session 11

4.6 Retention, deletion and backups 12

4.7 Requirements derived from a legal or regulatory instrument 12

4.8 Design authority with recorded decisions 13

4.9 Vendor platform versus owned policy 13

4.10 Assurance evidence for an external inspector 14

4.11 Suggested framing, in three sentences 15

4.12 Mapping to your own history 15

5. Vocabulary translation table 16

6. Common requirements — the BI solution and the CDA solution 17

Purpose

A CDA disclosure capability looks, from the outside, like an exotic regulated system that only telecoms people have touched. From the inside it is a data platform: sources feed a pipeline, the pipeline normalises and stores, a query layer answers questions under access control, and a governance function worries about quality, lineage and who saw what. That is the shape of every serious enterprise BI estate. The differences are real — the law is the requirements baseline, the data subject is a person under investigation, and over-delivery is a breach — but they are differences of consequence, not of structure.

This paper does three things. Section 1 maps the CDA reference architecture onto the standard BI stack, layer by layer. Section 2 lists the disciplines that transfer directly and names the vocabulary on each side so you can translate on the fly. Section 3 is the honest list of what does *not* transfer, because claiming too much is the fastest way to lose a telecoms audience. Section 4 turns it into things you can say about your own background, with prompts to fill in from your own history.

1. The same stack, different labels

The CDA course's six-layer reference architecture maps almost one-to-one onto a conventional analytics platform. The names differ; the responsibilities do not.

```
CDA reference architecture            Enterprise BI / data platform
--------------------------            -----------------------------
L6  External interface                Reporting / API / consumer-facing
L5  Disclosure management             Semantic layer, workflow, approval
L4  Query and resolution services     Query engine, master-data / MDM lookup
L3  Retention store                   Warehouse / lakehouse (immutable zone)
L2  Ingest and normalisation          ELT / ETL, staging, data quality gates
L1  Sources                           Operational systems of record
L0  Audit, time, IAM, keys, change    Platform engineering, data governance
```

CDA layer	BI equivalent	Same problem, different stakes
Sources (IMS nodes, HSS, NAT logs)	ERP, CRM, billing, event streams	Several systems describe one business event; nobody has nominated a system of record; a routine upgrade changes a field's meaning without telling anyone downstream
Ingest and normalisation	Staging, ETL/ELT, data quality framework	Parse, canonicalise identifiers, deduplicate, reject and quarantine bad records, reconcile volumes against source counts
Retention store	Warehouse, data vault, time-partitioned fact tables	Immutability, partitioning by load date, lifecycle and archival policy, backup retention that silently exceeds the stated policy
Query and resolution	MDM / golden record, dimensional lookups, query engine	Resolving a business key to the right entity as it was at a point in time — slowly changing dimensions are exactly the interval-binding problem
Disclosure management	Semantic layer, report approval, row-level security	Who may see which columns and rows, approval before release, caveats attached to figures
External interface	Dashboards, APIs, data sharing agreements	Structured outputs with provenance; consumers who take the number and lose the footnote
Cross-cutting	Data governance office, platform ops, security	Lineage, access review, change control, key management, monitoring

> **TALKING POINT**
>
> In a conversation: 'The CDA reference architecture is a regulated data warehouse with a legally defined consumer. Every layer has a direct BI counterpart; what changes is the cost of getting a row wrong.'

2. Disciplines that transfer directly

Each item below names the BI discipline, the CDA concept it corresponds to, and the sentence that connects them. These are the places where experience in business data analysis is not merely analogous but the same work.

2.1 Data cataloguing and classification

In BI	In CDA
Business glossary, data dictionary, data classification (public / internal / confidential / PII), ownership and stewardship	Field-level catalogue with entity / events / content classification, legal basis, retention period, source, owner and sign-off date
Tooling: Collibra, Alation, Purview, dbt docs, home-grown spreadsheets	The course's point: the catalogue must be a build artefact the pipeline consumes, not a governance folder

The transferable lesson is one every BI practitioner has learned painfully: a classification that lives only in a document drifts from the schema within a quarter. The CDA version has legal effect — an entity-only authorisation must be mechanically incapable of returning events data — so the catalogue-as-code argument is stronger, not different.

2.2 Master data, identity resolution and slowly changing dimensions

The single most consequential correctness requirement in CDA — every identifier lookup is only valid with respect to a time — is a Type 2 slowly changing dimension. A customer key that pointed at account A until February and account B afterwards is the same problem as a telephone number reassigned between subscribers. BI practitioners already know the failure mode: a join on the current value of a dimension rewrites history.

```
BI: customer_dim (customer_key, valid_from, valid_to, is_current)
CDA: binding_store (identifier, type, valid_from, valid_to)

Both: query AS AT a time; return the row in force; make gaps explicit.
```

Where CDA is stricter: MDM tools routinely use probabilistic matching (fuzzy name and address scoring) to build golden records. The course rules that out in the resolution path because nobody can explain a match score in a witness box. If you have built or governed an MDM programme, you know exactly where the heuristics live, which is precisely the audit a CDA design authority needs someone to run.

2.3 Data quality frameworks

The course's six quality dimensions — completeness, accuracy, timeliness, consistency, validity, uniqueness — are the DAMA DMBOK dimensions that every data quality framework uses. The control the course singles out, volume reconciliation against source-side counts, is standard warehouse practice: row-count reconciliation between source extract and loaded target, with a failure threshold. The vocabulary is identical.

Dimension	BI control	CDA control
Completeness	Source-to-target row counts; late-arriving-data monitoring	Volume reconciliation per feed per period; feed-outage register
Accuracy	Sample reconciliation to source; finance tie-outs	Sampled records verified against source records
Timeliness	SLA on load latency; freshness dashboards	Generation-to-availability latency, reported at the tail

Dimension	BI control	CDA control
Consistency	Cross-system reconciliation (e.g. GL vs sub-ledger)	Cross-node agreement for the same session
Validity	Schema and domain validation at staging	Schema and identifier-format validation at ingest
Uniqueness	Dedup on business key	Duplicate detection on ICID / session

> **TALKING POINT**
> 'A feed that stops produces silence, not errors, and silence looks like a quiet period. I have seen that in sales data; the CDA version means every disclosure in the window is potentially incomplete and nobody knows.'

2.4 Lineage and provenance

BI lineage answers 'where did this number on the board pack come from?' — source table, transformation, version of the model, load run. CDA provenance answers the same question for a court: which source produced this field, when, through which pipeline version. The course's provenance statement — *this field came from the S-CSCF record ingested at T under pipeline version 4.2* — is column-level lineage with a run identifier. Anyone who has implemented lineage capture, or argued for it against a tight budget, has done this.

2.5 Access control: row-level and column-level security

The CDA scope resolver intersects an authorisation with a request and emits a permitted field set and period. In BI terms that is column-level security (which fields) plus row-level security (which period, which subject), computed per query rather than per session. The design options the course ranks — procedural, in the UI, in a service beneath the application, in the data layer — are exactly the options every analytics platform weighs for sensitive data, and the course's conclusion (service beneath the application, with data-layer security for the most sensitive classes) is the pattern mature BI estates arrive at for PII and finance data.

2.6 Retention, archival and deletion

Every regulated BI estate has a records-retention schedule, time-partitioned fact tables, an archival tier and a deletion policy that the backup team has quietly never implemented. The course's findings — partitioning is the deletion mechanism, deletion must produce evidence, backups outlive the retention period, crypto-shredding is the realistic answer — are the GDPR right-to-erasure conversation that data platforms have been having since 2018.

2.7 Operational reporting versus evidential reporting

BI distinguishes exploratory analytics from financial reporting that will be audited. The first can tolerate approximation; the second needs a controlled, reproducible, signed-off number with a tie-out. The course's distinction between data used to direct an investigation and data used as evidence is the same split, and the design response — decide explicitly which outputs are built to the higher standard, and accept the cost — is the same conversation finance teams have about which reports are 'controlled'.

2.8 Capacity planning and the thing everyone forgets

The course's list of under-estimated capacity drivers (translation logs dominate; indexes as large as the data; steady state versus growth curve; a new population added mid-life) is the warehouse capacity conversation. Any BI architect has watched an event stream or a clickstream table dwarf the fact tables the plan was sized for, and has had to explain why the index footprint was omitted.

2.9 Vendor platforms versus owned policy

'Buy the plumbing, own the policy' is the BI platform argument in one line: the warehouse, the orchestration tool and the BI front end are commodity; the semantic model, the business rules, the classification and the access policy are the organisation's and must be portable. Anyone who has migrated between BI platforms and discovered that the business logic lived inside one vendor's product has learned the lesson the course is teaching.

2.10 Requirements traceability and governance

The interpretation chain — legal instrument to interpretation to capability to testable requirement, every link versioned — is a requirements traceability matrix. The design authority with a standing decision list, recorded rationale and a two-years-later retrieval test is an architecture review board run properly. Both exist in any large programme; the course merely insists they be run with more rigour because an inspector, not a steering committee, will ask for the record.

3. What does not transfer — say this first

Claiming that CDA is 'just BI' loses a telecoms or Home Office audience immediately. The credible position is: the engineering disciplines transfer; the following things are different in kind, and I know that.

Difference	Why it matters
The requirements baseline is a legal instrument	A notice is written to be legally effective, not implementable. It must be translated through a governed interpretation, and the translation is re-validated when the law changes. BI requirements come from a product owner who can be asked what they meant.
The data subject is a person under investigation	Errors are not misreported revenue; they are wrongful disclosures about real people, reportable to an oversight body and actionable at the Tribunal.
Over-delivery is a breach	BI rewards collecting more. Here, retaining a field the notice did not specify, or returning more than the authorisation covers, is a compliance failure. The instinct of every data professional — keep it, it might be useful — is the wrong instinct.
Minimisation is a design goal	Negative decisions (what was deliberately not built) are recorded as evidence. BI backlogs do not usually carry a 'we chose not to' register.
The content boundary	There is a category of data — content — that the capability must be demonstrably incapable of disclosing. BI has sensitivity tiers; it does not have a category whose presence is itself unlawful under the power being exercised.
Misuse looks exactly like use	A BI query run for curiosity is a policy breach; a CDA query run for a personal reason is a criminal offence (s.11) that is technically indistinguishable from an authorised one. Detection controls — self-query alerting, reconciliation with the requesting authority — have no real BI equivalent.
Explicability is a veto	BI embraces ML scoring and probabilistic matching. In the CDA resolution and correlation path they are ruled out because a witness must explain the answer.
The consumer has statutory standing	The acquiring authority is not a stakeholder to be satisfied; it is a body exercising a power, and the operator's duty to it is fixed by law, not by service agreement.
The lifetime	A retention capability outlives the network generation that fed it. Few BI platforms are designed on the assumption that every source will be replaced.

TALKING POINT

'I would not describe this as BI. I would say that the engineering disciplines a CDA capability depends on — cataloguing, master data, quality reconciliation, lineage, row and column security, retention — are ones I have run in a less forgiving domain's cousin, and what I needed to learn was the law, the taxonomy and the asymmetry of failure.'

4. Presenting your own experience — solutions

The strongest claim is not 'I have done BI' but 'I have designed and governed large consumer-facing systems where data about individuals was processed under a regulatory regime, and the specific disciplines a CDA capability depends on were part of that work'. For each discipline this section states what needs to be done in one line, then gives a solution — the mechanism, with its components named — as it was built or could have been built on one of your programmes, and then the CDA form of the same solution. The solutions are written so you can describe them, draw them on a whiteboard, and defend the design choices.

> **BEFORE YOU USE THESE**
>
> The solutions are drawn from the roles and systems you have described — HMCTS Common Platform, the Scottish Water integrations, the HMRC RESTRICTED wireless network, trainline — and are written as a competent architect would have solved the problem in that environment. Where a detail does not match what was actually built, change it to what was, or present it as 'what I would build'; both are credible, and the second is often the stronger answer in a design-governance interview.

4.1 Field-level classification with legal effect

What needs to be done: every data item carries a classification the system enforces, so that who may see it is a property of the data, not of the document that describes it.

Solution — courts case management (Common Platform)

- **Marking as data.** Each case data item (party details, addresses, prosecution material, youth status, reporting restrictions, special measures) carries a sensitivity marking stored as an attribute on the record in the case store, assigned at creation by the originating service according to a published marking scheme owned by the design authority.
- **One policy decision point.** A central authorisation service (a policy decision point) evaluates every read against three inputs: the record's marking, the requesting user's attributes (organisation, role, relationship to the case), and the context (which case, which hearing). Services do not implement their own access logic; they call the PDP and receive an allow/deny with the permitted field set.
- **Enforcement at the API gateway and in the data layer.** The gateway strips fields the PDP did not permit before the response leaves the service; the data layer applies a view filter for the most sensitive markings so a bug in the service cannot leak them.
- **Marking scheme as code.** The marking scheme is a versioned configuration artefact consumed by the PDP, with a change process requiring design-authority sign-off; a test suite asserts that each role sees exactly the documented field set for each marking.
- **Audit per decision.** Every PDP decision is logged with the inputs, so a disclosure question later can be answered from the decision log.

The CDA form

The field-level catalogue is the marking scheme; entity / events / content is the marking; the scope resolver is the PDP, taking the authorisation as the 'relationship to the case'; the disclosure gateway strips un-permitted fields; the data layer separates entity and events stores physically. The requested / authorised / returned triple is the decision log.

4.2 Time-qualified identity resolution

What needs to be done: resolve an identifier to the right entity *as at* a time, with history and gaps preserved, so that a historical record is never attributed to whoever holds the identifier today.

Solution — party identity across cases (Common Platform) and asset identity across systems (Scottish Water)

- **A person index with interval-valid identity records.** A dedicated party index service holds, per person, the set of external identifiers (police reference, CPS reference, court reference, PNC identifier) and the identity attributes (name, date of birth, address), each row carrying valid-from / valid-to and a recorded-at timestamp — a bitemporal record. Nothing is overwritten; a change inserts a new interval and closes the old one.
- **Deterministic linking only.** A new reference is linked to a person only on an exact match of a governed key (e.g. PNC identifier plus date of birth); candidate matches that fall short go to a stewardship queue for a human decision, which is recorded with the reason. No scored matching in the automated path.
- **As-at queries by default.** The index API takes a timestamp on every lookup and returns the interval in force; a lookup over a period returns the segmented list, including explicit 'no identity held' gaps. A current-value lookup is a separate, deliberately named call.
- **Integration mapping tables with validity.** On the Scottish Water integrations, the mapping between Maximo asset and work-order identifiers and the operational systems' identifiers is held in a mapping table with effective dates, so a work order replayed for a past period resolves to the identifier that was in force then.
- **Correction with audit.** A wrong link is corrected by closing the interval and recording the correction; the previous resolution remains queryable, so any output produced under it can be found.

The CDA form

The binding store: MSISDN, IMSI, IMEI, IMPU and account, each an interval with valid-from / valid-to and recorded-at. Resolution returns time-segmented results with 'not allocated' gaps; identifier-to-identifier hops are bounded per authorisation and counted in the audit; the stewardship queue is the human review point and 'insufficient information' nil return.

4.3 Volume reconciliation and silent feed loss

What needs to be done: detect that a feed has stopped or thinned when the only symptom is that fewer records arrived, by comparing against an expectation the pipeline itself does not produce.

Solution — integration platform (Scottish Water work-order integrations)

- **Source-side counters.** Each producing system emits, on a schedule, a control record stating how many work orders it sent in the period (a count-and-checksum message on its own topic, written by the producer, not by the integration layer).
- **Target-side counters.** Each function app records what it received, accepted and rejected per period in a reconciliation table, keyed by feed and period.
- **A reconciliation job.** A scheduled job compares producer count, received count and accepted count per feed per period; mismatches above a threshold raise an alert to the integration team and the consuming business owner, with the delta. Rejected records are listed with their reasons, never discarded.
- **A baseline for feeds that cannot count.** Where a producer cannot emit a control record, the job compares the period's volume with a rolling baseline for that weekday and hour and alerts on a statistically significant drop — including a drop to zero, which is the case a count match would hide.
- **A heartbeat independent of data.** Each feed sends a heartbeat even when it has nothing to send, so 'no records' and 'no feed' are distinguishable.
- **Feed-outage register.** Every detected outage is recorded with start, end and affected period, so downstream reports can carry a completeness note for the window.

The CDA form

Identical. Source-side counts from each IMS node, the NAT platform and the HSS feed; expected-versus-actual per feed per hour; a heartbeat per feed; a feed-outage register that generates the completeness statement attached to every disclosure covering the window. This is the single control the course puts first.

4.4 Lineage and provenance for a scrutinised output

What needs to be done: for any item in an output that will be challenged, show which source produced it, through which version of which logic, when — without reconstructing from application logs.

Solution — court records and results (Common Platform)

- **Event-sourced case record.** Changes to a case are stored as an append-only sequence of events (hearing listed, plea entered, result recorded), each carrying the originating service, the user, the timestamp and the version of the service that wrote it. The current state of the case is a projection of the events; the events are the record.
- **Provenance on the projection.** Each field in the projected record carries a reference to the event that last set it, so a question about any displayed value resolves to its event in one lookup.
- **Immutable, signed event store.** Events are hashed in sequence; the store is append-only with the hash chain verified on a schedule, so an alteration is detectable.
- **Versioned transformation logic.** The projection code and any derivation (e.g. a result code mapped to a sentence description) is versioned, and the version is recorded on the projected field, so an output can be reproduced by replaying events through the same version.
- **Produce-on-demand.** A 'provenance report' endpoint takes a case and a field and returns the chain: event, user, service version, time — the artefact handed over when a record is challenged.

The CDA form

Retained records are the event store; the assembled disclosure is the projection; each disclosed field carries source node, feed, pipeline version and ingest time; partitions are hashed and signed; the provenance report is the per-disclosure evidence pack. Repeatability is replaying the same query against the same partitions under the same pipeline version.

4.5 Access control per request, not per session

What needs to be done: decide each request on the relationship between the requester and the specific thing requested, so that a valid role with no standing on this case — or no live authorisation — can do nothing.

Solution — case-based entitlement (Common Platform)

- **Relationship as data.** Each case holds its participant list — which organisations and which individuals are prosecution, defence (per defendant), court staff, judiciary — as structured data maintained by the services that assign them (e.g. a defence representative is added when a legal aid or representation order is recorded).
- **Per-request policy evaluation.** The PDP (4.1) evaluates each request with the case identifier as an input and looks up the requester's relationship on that case; role alone never grants access to case content. Session tokens carry identity and role, not entitlements; entitlements are computed per call.
- **Scope objects.** The PDP returns a scope object — permitted fields, permitted defendants, permitted document classes — that the service and gateway apply mechanically, so the policy and the enforcement are separated and the policy can be tested on its own.
- **Standing removed automatically.** When a representative is withdrawn from a case, the relationship row is closed and their access ends on the next request without any account change.

- **Break-glass with review.** A narrow emergency path (e.g. urgent out-of-hours listing) allows a named role to act outside normal entitlement; every use is flagged in the audit and reviewed the next working day.

The CDA form

The authorisation is the relationship row: machine-readable, with type, purpose, data classes, period and expiry. The scope resolver returns the scope object (field set, period, subject); accreditation is an attribute checked on each request; expiry of the authorisation ends access with no account change; the urgent path is the break-glass with mandatory next-day review.

4.6 Retention, deletion and backups

What needs to be done: delete at the end of the retention period in a way that can be evidenced, including from backups, with legal hold as a designed exception.

Solution — records retention on a government case system

- **Retention class on every record.** Each record carries a retention class (from the records retention schedule) and a trigger date (e.g. case closure); the retention engine computes the expiry date from the class and the trigger.
- **Time-partitioned storage by expiry.** Data is stored in partitions keyed by expiry month; expiry is a partition drop, not a row-by-row delete, so it is fast, complete and verifiable by partition count.
- **Envelope encryption with per-partition keys.** Each partition is encrypted under its own data key, wrapped by a key in the HSM-backed key management service. Backups copy the encrypted partitions; the keys are never in the backup set. Deleting a partition destroys its key in the KMS, so the backup copy is unreadable from that moment — no backup rewriting required.
- **Deletion certificate.** The retention engine writes a signed deletion record (partition, expiry date, key destruction timestamp, operator) to an audit store with a longer retention than any data, and a monthly report lists partitions due, deleted and held.
- **Legal hold as a first-class state.** A hold is a record against a partition or a subject with a reason, an owner and a review date; the engine skips held partitions and reports them; a hold cannot be placed or lifted without two approvals.
- **Derived copies in the inventory.** Indexes, reporting extracts and caches are registered as derived datasets with their own expiry bound to the source partition, so the drop cascades.

The CDA form

Partitions keyed to event time with the notice period as the class; crypto-shredding at expiry; deletion evidence retained longer than the data; legal hold with two-person approval; the correlated store, indexes and any analytics extract registered as derived copies that expire with the source. This is the course's Module 7 as a built system.

4.7 Requirements derived from a legal or regulatory instrument

What needs to be done: turn legal text into testable requirements through a recorded interpretation, so that each requirement cites the interpretation and a change in the law re-validates the chain.

Solution — procedure-rule-driven requirements (Common Platform)

- **Instrument register.** The Criminal Procedure Rules, Practice Directions and the statutory provisions that set what the system must record and serve are held in a register, each with its version and effective date.
- **Interpretation records.** For each relevant provision, an interpretation record states in plain language what the programme reads it to require (e.g. 'a notice served under rule X means: the system must record the

time of service, the method, the recipient, and retain evidence of delivery'), who approved that reading (legal and the design authority), and when. Requirements cite the interpretation record, never the rule directly.

- **Traceability in the requirements tool.** Each requirement links upward to an interpretation and downward to the tests that prove it, so 'show me everything that depends on rule X' is a query.
- **Change trigger.** When a rule or direction is amended, the instrument register raises a review on every interpretation that cites it; the design authority re-approves or revises, and the linked requirements and tests are re-run.
- **Negative decisions recorded.** Where a provision was read as not requiring a capability, that reading is recorded too, so the absence can be explained later.

The CDA form

The notice and the Code of Practice are the instrument register; the interpretation record sits between each data description in the notice and the retained schema; requirements cite interpretations; a varied notice or a new Code triggers re-validation; the negative-decision register is the evidence of minimisation. The same traceability tool can hold it.

4.8 Design authority with recorded decisions

What needs to be done: make design governance a decision-taking body whose decisions can be produced years later with rationale and date.

Solution — architecture governance on a multi-year programme

- **Standing decision list.** The design authority maintains a list of the decisions it owns — identity model, access enforcement point, audit scope, marking scheme, integration patterns, data retention classes — each with a status (open, decided, superseded).
- **Architecture decision records.** Every decision is an ADR: context, options considered, decision, consequences, date, approver; stored in version control with the design, not in a meeting-minutes folder.
- **Decision-first agendas.** Each meeting takes the open decisions in order; document review happens only as input to a decision. A paper with no decision attached is not on the agenda.
- **Compliance by reference.** Design reviews check a design against the ADRs it must comply with, by reference, so a departure is visible as a departure from a numbered decision.
- **The two-year test.** Periodically a decision is picked at random and the record produced; failure is a governance finding.

The CDA form

The course's standing decision list — classification of arguable fields, truncation point, copy-versus-system-of-record, scope mechanism, correlation key, hop bound, urgent-path scope, human review rule, nil-return vocabulary, system-of-record nominations, vendor access model, notifiable-change criteria — kept as ADRs, with the inspector as the audience of the two-year test.

4.9 Vendor platform versus owned policy

What needs to be done: keep the behaviour you are accountable for in configuration you own and can move, so another party's change process and a supplier's roadmap cannot alter it unseen.

Solution — integration layer owned by another team (Scottish Water)

- **Contract-first interfaces.** Each integration is defined by a versioned contract (OpenAPI for the APIM-fronted function apps; schema for the Maximo messages) owned by the consuming programme,

with consumer-driven contract tests that run in the integration team's pipeline. A change that breaks the contract fails their build, not your deployment.

- **Business rules outside the platform.** Mapping, validation and routing rules live in configuration in your repository, loaded by the function app at start-up, not in the integration platform's own rule engine; a platform migration moves the configuration, not the logic.
- **Pinned platform dependencies and a canary.** The function app pins the runtime and shared-template versions it was tested against; a canary deployment of any integration-layer change runs your contract tests before it reaches production.
- **Change notification as a gate.** The integration team's change process includes a mandatory impact question against the registered consumers; a change affecting a contract cannot be released without the consumer's sign-off.
- **Exit plan written at entry.** Data formats, credentials (the Key Vault-based credentials management design) and configuration are documented as portable from day one.

The CDA form

Classification, scope rules, correlation logic, nil-return vocabulary, caveat model and audit schema in operator-owned configuration consumed by the vendor platform; ETSI-standard interfaces as the contract; vendor changes gated by the capability-impact question; retained-data portability and schema ownership in the contract; break-glass-only production access.

4.10 Assurance evidence for an external inspector

What needs to be done: produce evidence that each claimed control operates, on demand, to someone who was not in the room when it was designed.

Solution — CESG accreditation of a RESTRICTED wireless network (HMRC)

- **Security case with a control register.** The risk assessment produced a control register: each risk, the control that treats it, the evidence that proves the control operates, and the owner. The register was the accreditation document; the design existed to satisfy it.
- **Demonstrable, not asserted, controls.** For each control the evidence was defined before build: configuration exports showing the wireless segmentation and encryption settings; key-management procedures with signed key ceremonies; penetration test reports against the accredited build; access logs showing only accredited devices authenticated.
- **Evidence produced from the system, not written about it.** Where possible, evidence was generated by the system itself on a schedule — configuration snapshots, log extracts, certificate inventories — so the accreditor could request the current state, not last quarter's narrative.
- **Pre-accreditation rehearsal.** Before the accreditor's review, the team ran the evidence request against the register and recorded what could not be produced; each gap became a work item. The findings the accreditor raised were the ones left unresolved from the rehearsal.
- **Re-accreditation on change.** Any change to an accredited component triggered an impact assessment against the control register, so the security case stayed true to the built system.

The CDA form

The assurance evidence pack: control register mapping each IPA, TSA, UK GDPR and PECR obligation to a control and to the evidence the system produces monthly — requested/authorised/returned per sampled disclosure, reconciliation reports, deletion certificates, access reviews, self-query alerts, test reports. Rehearsed on test data before go-live; what cannot be produced is a design gap. Re-run on every notifiable change.

4.11 Suggested framing, in three sentences

1. **The structure is familiar.** 'A CSP disclosure capability is a six-layer data platform — sources, ingest, retention store, resolution and query, disclosure workflow, external interface — and I have designed and governed each of those layers in large consumer-facing systems.'
2. **The disciplines are the same.** 'The things that make it work are the same things that make any regulated data estate work: a catalogue the system actually enforces, time-qualified master data, reconciliation that catches silent feed loss, lineage, per-request access control, and deletion that can be evidenced.'
3. **The differences are the point.** 'What is different is that the requirements come from a legal instrument, the data subject is a person under investigation, and over-delivery is as much a breach as under-delivery. Those are the things a design authority has to hold in view, and they are why I approach it as governance rather than as a data project.'

4.12 Mapping to your own history

From what you have described of your background, the closest analogues are these; use them as starting points, not as scripts.

Your environment	Closest CDA parallel	Line of argument
Criminal courts case-management (HMCTS Common Platform)	Evidential weight, chain of custody, access that depends on the case rather than the role; data about individuals in proceedings	Records whose integrity would be tested in court; entitlements driven by matter and party; disclosure rules about what may leave the system and to whom
Regulated utility operational data (Scottish Water work-order integrations)	Sources with no nominated system of record; reconciliation across integration function apps; change control on feeds between enterprise systems	Several systems describing one event; what breaks when a feed changes shape; who must be consulted before a release
Protectively marked government network (HMRC RESTRICTED wireless)	Compartmentation, accreditation, assurance evidence for an external accreditor, key management	Security case built to be inspected, not asserted; controls that had to be demonstrable
High-volume consumer platform (trainline.com)	Capacity drivers that dwarf the planned ones; identity and account data at scale; operational versus audited reporting	Sizing for the event stream, not the transaction table; customer identity across channels

> **GOVERNANCE NOTE**
>
> Keep the parallels at the level of discipline and consequence. Do not claim telecoms protocol knowledge you do not have; claim the governance and data-platform knowledge you do, and be explicit that the protocol detail is what the reference annex is for.

5. Vocabulary translation table

For use in real time. When someone from the BI side says the left column, the CDA term is on the right, and vice versa.

BI / data platform term	CDA term
Data dictionary / business glossary	Field-level catalogue
Data classification tier (PII, confidential)	Entity / events / content classification
Golden record, MDM, party master	Identity resolution; binding store
Slowly changing dimension (Type 2)	Interval-keyed bindings; time-segmented resolution
Surrogate key / business key	ICID (charging correlation identifier); MSISDN / IMSI / IMPU
Source-to-target reconciliation	Volume reconciliation against source counts
Lineage	Provenance
Row-level / column-level security	Scope enforcement; permitted field set and period
Semantic layer / certified dataset	Disclosure management; caveated, assembled result
Controlled report / audited figure	Evidential-standard disclosure
Data retention schedule	Retention notice period; expiry and deletion evidence
Right to erasure	Destruction at end of period, including backups
Data owner / steward	Design authority decision-maker; system-of-record nomination
Architecture review board	Design authority with a standing decision list
Requirements traceability matrix	Legal instrument to interpretation to capability to requirement chain
Data sharing agreement	Data supply agreement (per source); capability statement (to the authority)
Null / not found	Nil-return vocabulary: not our subscriber / not allocated / no records / insufficient information / outside scope

6. Common requirements — the BI solution and the CDA solution

Requirements that recur in almost every data platform, stated once, with the standard enterprise BI answer and the CDA answer alongside. Where the answers are the same, say so; where CDA is stricter, the third column says why.

Requirement	Enterprise BI solution	CDA solution	Why CDA differs
Bring data from many sources into one queryable store	ETL/ELT pipelines into a warehouse or lakehouse; staging then curated layers; orchestrated and monitored	Ingest from IMS, core, NAT and business systems into a dedicated retention store with truncation and classification at ingest	The source deletes its copy (PECR); ingest must be right first time and rejects must be counted
Know what every field means and how sensitive it is	Data catalogue with business glossary and classification tags; stewardship	Field-level catalogue with entity / events / content classification and sign-off, consumed by the pipeline and the access layer	Classification has legal effect; it must be enforced, not documented
Resolve an identifier to the right entity	MDM golden record with cross-reference; deterministic or probabilistic matching	Binding store with interval validity; deterministic only; hop-bounded; explicit gaps	Each hop is a disclosure; matching must be explicable in court
Answer questions about the past correctly	Type 2 slowly changing dimensions; as-at queries	Time-segmented resolution; bitemporal corrections	A wrong as-at answer attributes someone else's communications to a person
Join records of one event from several systems	Conformed dimensions and surrogate keys; business-key matching	Correlation on ICID; never time-and-identifier heuristics	A 99% join silently produces wrong disclosures in the 1%
Know that all the data arrived	Source-to-target row counts; control totals; freshness alerts	Volume reconciliation per feed per period against an independent expectation; completeness statement per disclosure	Silence looks like a quiet period; every disclosure in the window is potentially incomplete
Ensure the data is accurate	Data quality rules; sampling against source; scorecards	Same, plus tail-latency reporting and accuracy statements on timestamps and location; caveats that travel with the data	The receiving controller must distinguish fact from assessment (DPA 2018 s.38)
Control who sees what	RBAC plus row-level and column-level security; masking; query audit	Attribute-based scope enforcement per request with the authorisation as input; field set and period; human release approval	A valid role with no live authorisation must be able to do nothing
Prevent misuse by insiders	Separation of duties; privileged access management; query logging	Compartmentation of provisioning, data, disclosure and audit; two-person control; self-query and associate-query alerting; reconciliation with the authority's records	Misuse is technically identical to use; detection is mandatory
Prove where a number came from	Column-level lineage with run identifiers	Provenance per field: node, feed, pipeline version, time; repeatable queries	The audience is a court, not an analyst

Requirement	Enterprise BI solution	CDA solution	Why CDA differs
Keep data only as long as allowed	Retention schedule; partition lifecycle; archival tiers	Partitions keyed to event time; expiry per data family; deletion evidence retained longer than the data	Over-retention is a breach under IPA, UK GDPR and PECR
Delete from backups too	Usually unsolved; backups age out	Crypto-shredding: per-partition keys destroyed at expiry; legal hold as a designed exception	s.92 IPA requires destruction of every copy
Handle requests from the people the data is about	Subject-access process; erasure and rectification workflows	Defined route with exemptions applied per request (DPA Sch 2 paras 2, 5); neither-confirm-nor-deny wording; audit queryable by subject without creating a disclosure	Revealing a disclosure is itself unlawful; refusing ordinary account data is also unlawful
Serve data to consumers	Dashboards, APIs, extracts; data sharing agreements	Structured, authenticated, audited interface (ETSI TS 103 120 / 102 657) with machine-readable authorisation, clarification messages and a nil-return vocabulary	The consumer is exercising a statutory power; ambiguity belongs with them
Deal with urgent requests	Priority queues; SLA tiers	A narrow pre-agreed urgent path; human review moved after release, not removed; visibly distinct in audit	Threat to life versus the evidential standard — both must survive
Change the platform safely	CI/CD, regression tests, change advisory board	Same, plus a mandatory capability-impact question on every change and a notifiable-change decision (IPA s.258A)	A tariff change can silently alter a retained field
Use a vendor platform	Buy the platform, configure it, own the semantic model	Buy the plumbing, own the policy: classification, scope, correlation and audit schema in operator-owned configuration; Art 28 terms; break-glass only; portability at exit	Losing portability means being unable to meet the notice with another supplier
Plan capacity	Forecast from source growth and consumers; include indexes	Measure the NAT-record-to-session ratio; include correlated copies, replicas and indexes; steady state at twelve months; IoT populations	The biggest dataset is the one nobody sized for
Monitor the platform	Data observability: freshness, volume, schema, lineage	Same, plus security-log retention of 13 months (TSA Code) and misuse detection	Three regulators look at the same logs
Respond to incidents	Runbooks, on-call, post-incident review	Error taxonomy with detection, enumeration of affected past disclosures, and pre-agreed notification routes: IPCO, Information Commission (72h / 24h PECR), Ofcom	An error you cannot trace to past disclosures is not remediated
Show an auditor you comply	Audit evidence: policies, access reviews, test results	Assurance pack produced monthly from controls in place; rehearsed on test data before go-live; requested/authorised/returned per sampled disclosure	What cannot be produced is a design gap

Requirement	Enterprise BI solution	CDA solution	Why CDA differs
Govern decisions	Architecture review board; data governance council; ADRs	Design authority with a standing decision list, recorded rationale, and a two-years-later retrieval test; negative-decision register	An inspector, not a steering committee, will ask for the record
Define what the platform cannot do	Rarely done explicitly	Capability statement acknowledged by the authority; data availability matrix per request type	The commonest CDA failure is a requirement the data could never satisfy